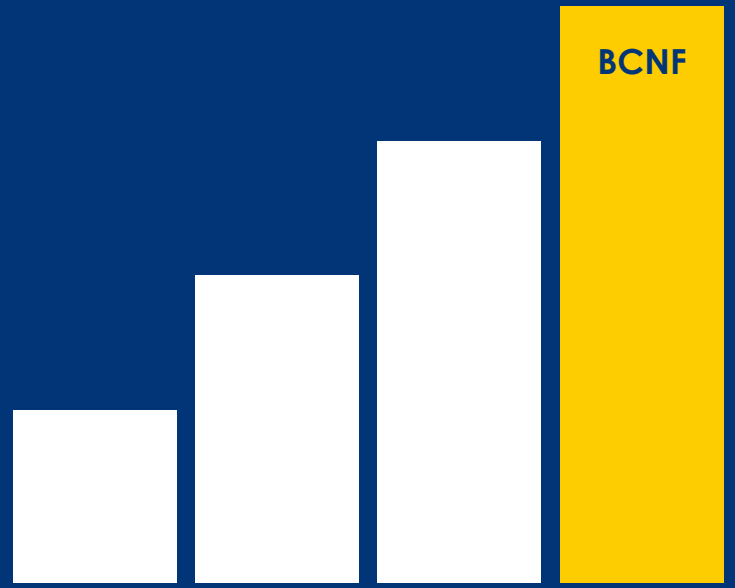
ISC-321 · FUNDAMENTOS DE BASES DE DATOS

Normalización, parte 1

Dependencias funcionales, 1FN, 2FN, 3FN y BCNF

Elmasri & Navathe, cap. 10.1-10.5



La ruta de hoy



Diseño informal

Cuatro directrices y las anomalías que evitan

Dependencias funcionales

DF, reglas de Armstrong, clausura X^+ y cobertura mínima

1FN · 2FN · 3FN

Pruebas y descomposiciones basadas en las claves

BCNF

Todo determinante es superclave

Meta práctica: llegar a **3FN o BCNF**. 1FN y 2FN son pasos intermedios.

1

Diseño informal

¿Cómo distinguir un buen esquema de relación de uno malo?

¿Por qué normalizar?

- Hasta ahora la calidad dependía de la **intuición** del diseñador
- La normalización da una forma **formal** de medir si un esquema es bueno

Dos niveles de calidad

Lógico: los usuarios entienden el significado de los atributos
Ej.: en `EMPLEADO` es obvio qué es `FechaNac`

Implementación: cómo se almacenan y actualizan las tuplas
Ej.: guardar `NombreDpto` una sola vez, no por empleado

Dos metodologías

Ascendente (bottom-up, síntesis): parte de relaciones entre atributos individuales

Descendente (top-down, análisis): parte de agrupaciones naturales (facturas, formularios, informes) y las descompone
Ej.: una factura → `CLIENTE`, `FACTURA`, `LÍNEA_FACTURA`

Esquema de trabajo: EMPRESA simplificada

```
EMPLEADO(NombreE, Dni, FechaNac, Dirección, NúmeroDpto)
DEPARTAMENTO(NombreDpto, NúmeroDpto, DniDirector)
LOCALIZACIONES_DPTO(NúmeroDpto, UbicaciónDpto)
PROYECTO(NombreProyecto, NumProyecto, UbicaciónProyecto, NumDptoProyecto)
TRABAJA_EN(Dni, NumProyecto, Horas)
```

Diseño bueno

Cada relación representa **una** entidad o **un** vínculo.

Lo que vamos a romper

`EMP_DEPT` y `EMP_PROY` mezclan entidades para ver qué sale mal.

Cuatro directrices informales

1

Semántica clara

No mezclar atributos de varias entidades o vínculos en una sola relación

Ej.: `EMP_DEPT` guarda empleado **y** departamento en la misma fila
Evita: Ambigüedad

2

Sin anomalías

Sin anomalías de inserción, borrado o modificación

Ej.: Cambiar el director del dpto. 5 en `EMP_DEPT` obliga a editar **cada** empleado del dpto.
Evita: Redundancia e inconsistencia

3

Evitar NULL

Pocos valores NULL frecuentes en relaciones base

Ej.: `NúmeroOficina` en `EMPLEADO` queda en NULL para el 90 % sin oficina
Evita: Espacio, JOIN y agregados confusos

4

Sin tuplas falsas

Concatenar solo por pares clave principal – foreign key

Ej.: Unir `EMP_LOCS` y `EMP_PROY1` por `UbicaciónProyecto` inventa filas
Evita: Filas inventadas

Directriz 2: redundancia y anomalías

NombreE	Dni	NúmeroDpto	NombreDpto	DniDirector
Pérez Pérez, José	123456789	5	Investigación	333445555
Campos Sastre, Alberto	333445555	5	Investigación	333445555
Jiménez Celaya, Alicia	999887777	4	Administración	987654321
Sainz Oreja, Juana	987654321	4	Administración	987654321

EMP_DEPT: los datos del dpto. 5 se repiten

A veces se viola **a propósito** por rendimiento: documentarlo y controlarlo con triggers o procedimientos.
Mejor: relaciones base sin anomalías + **vistas** con los JOIN frecuentes.

Inserción

Nuevo empleado → repetir los datos del dpto. sin errores. Nuevo dpto. sin empleados → NULL en la clave `Dni`

Borrado

Borrar al último empleado de un dpto. → se pierde el departamento

Modificación

Cambiar el director del dpto. 5 → actualizar todas sus tuplas o queda inconsistente

Directrices 3 y 4: NULL y tuplas falsas

NULL puede significar…

– el atributo **no aplica**
– el valor es **desconocido**
– se conoce pero **aún no se ha grabado**

Si solo el 10 % de los empleados tiene oficina, crear `OFICINAS_EMPS(DniEmpleado, NúmeroOficina)` en vez de agregar `NúmeroOficina` a `EMPLEADO`.

Descomponer EMP_PROY mal

```
EMP_LOCS(NombreE, UbicaciónProyecto)
EMP_PROY1(Dni, NumProyecto, Horas,
          NombreProyecto, UbicaciónProyecto)
```

- Al unirlas por `UbicaciónProyecto` aparecen **tuplas falsas**: filas que nunca existieron
- Causa: `UbicaciónProyecto` no es clave principal ni foreign key en ninguna
- Partir bien = **reunión no aditiva** (cap. 11)

2

Dependencias funcionales

La herramienta formal para medir la calidad de un esquema

¿Qué es una dependencia funcional?

X → Y

si conozco X, conozco Y

- Dos filas con el mismo X **siempre** tienen el mismo Y
- Si X es **clave candidata** ⇒ X → Y para cualquier Y
- X → Y **no** implica Y → X
- Sale del **significado** de los datos, no de lo guardado hoy
  - No se confirma mirando la tabla; **un contraejemplo** la descarta

DF de EMP_PROY

```
DF1: {Dni, NumProyecto} → Horas
DF2: Dni → NombreE
DF3: NumProyecto →
       {NombreProyecto,
        UbicaciónProyecto}
```

Con el `Dni` sé el `NombreE`, pero para las `Horas` necesito **las dos**: `Dni` y `NumProyecto`.

Contraejemplo: IMPARTIR

Profesor	Curso	Texto
Smith	Estructuras de datos	Bartram
Smith	Administración de datos	Martin
Hall	Compiladores	Hoffman
Brown	Estructuras de datos	Horowitz

Texto → Curso

Parece cumplirse, pero un solo estado no basta para confirmarlo.

Profesor → Curso

No se cumple: Smith imparte dos cursos. Un contraejemplo basta.

Reglas de inferencia

Regla	Nombre	Enunciado	Ejemplo
RI1	**Reflexiva**	Si $X \supseteq Y \Rightarrow X \rightarrow Y$ (trivial)	{Dni, NombreE} → NombreE
RI2	**Aumento**	$X \rightarrow Y \Rightarrow XZ \rightarrow YZ$	Dni → NombreE ⇒ {Dni, NumProy} → {NombreE, NumProy}
RI3	**Transitiva**	$X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$	NúmDpto → DniDir, DniDir → TelDir ⇒ NúmDpto → TelDir
RI4	Descomposición	$X \rightarrow YZ \Rightarrow X \rightarrow Y$	NumProy → {NombreProy, UbicProy} ⇒ NumProy → NombreProy
RI5	Unión	$X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$	Dni → NombreE, Dni → Dirección ⇒ Dni → {NombreE, Dirección}
RI6	Seudotransitiva	$X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$	NombreE → Dni, {Dni, NumProy} → Horas ⇒ {NombreE, NumProy} → Horas

Resaltadas: axiomas de Armstrong. Notación: {X, Y} → Z se escribe XY → Z.

Axiomas de Armstrong: sólidos y completos

Sólidos

Todo lo que se deduce con ellos es verdad: **nunca** producen una DF falsa.

Completos

Aplicándolos una y otra vez se obtienen **todas** las DF de F^+.

RI4-RI6

No agregan poder: se demuestran con RI1-RI3. Son **atajos**.

Cuidado, esto NO se puede inferir

$X \rightarrow A$ y $Y \rightarrow B \not\Rightarrow XY \rightarrow AB$

$XY \rightarrow A \not\Rightarrow X \rightarrow A$ ni $Y \rightarrow A$

Clausura de un conjunto de atributos: X^+

```
Algoritmo 10.1
X⁺ := X
repetir
    antiguaX⁺ := X⁺
    por cada Y → Z en F:
        si Y ⊆ X⁺ entonces X⁺ := X⁺ ∪ Z
hasta que X⁺ = antiguaX⁺
```

Si X^+ trae **todos** los atributos, X es **clave** (superclave).

Con las DF de EMP_PROY

X	X^+
{Dni}	{Dni, NombreE}
{NumProyecto}	{NumProyecto, NombreProyecto, UbicaciónProyecto}
{Dni, NumProyecto}	**todos** los atributos → es clave

Equivalencia y cobertura mínima

- **F cubre a E**: toda DF de E se deduce de F
- **E y F equivalentes**: $E^+ = F^+$
- **Conjunto mínimo**:
  - un **solo atributo** a la derecha
  - sin atributos **sobrantes** a la izquierda
  - sin DF **redundantes**

Ejemplo: $E = \{B \rightarrow A,\ D \rightarrow A,\ AB \rightarrow D\}$

1. Ya están en forma canónica (un atributo a la derecha)

2. $B \rightarrow A \Rightarrow$ (aumento) $B \rightarrow AB$; con $AB \rightarrow D \Rightarrow B \rightarrow D$. Se reemplaza $AB \rightarrow D$ por $B \rightarrow D$

3. $B \rightarrow D$ y $D \rightarrow A \Rightarrow B \rightarrow A$ es **redundante**

Cobertura mínima: $\{B \rightarrow D,\ D \rightarrow A\}$

Claves y atributos primos

Superclave

Columnas que identifican una fila (puede sobrar alguna). `{Dni, NombreE}`

Clave

Superclave sin nada de sobra: si quitas una columna, ya no identifica. `{Dni}`

Clave candidata

Cualquiera de las claves. Una es la **principal**; el resto, **secundarias**

TRABAJA_EN

Dni	NumProyecto	Horas
primo	*primo*	*no primo*

Atributo primo

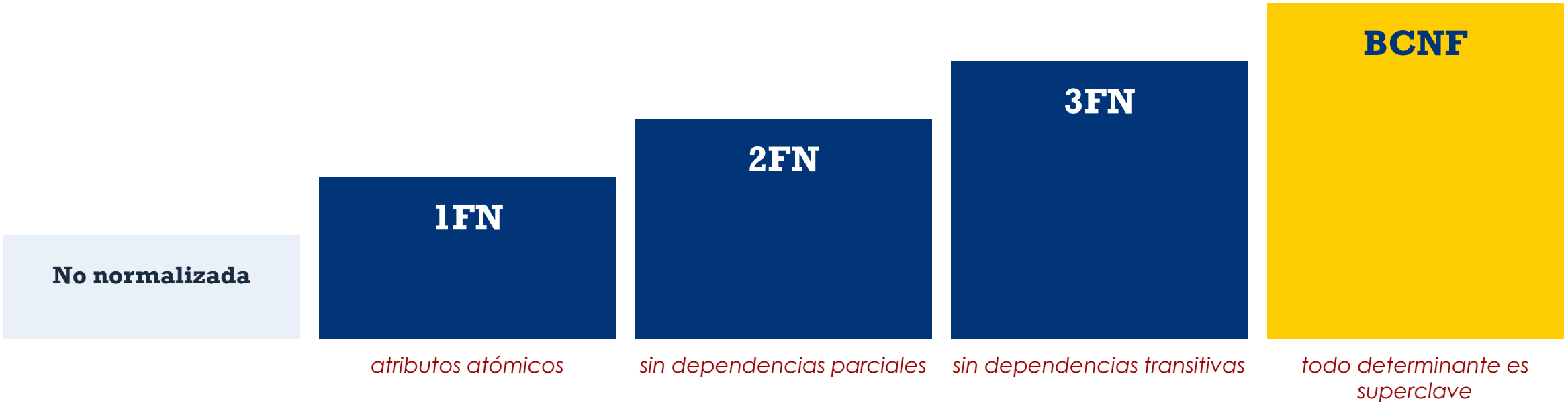
Forma parte de **alguna** clave candidata. El **no primo** es un dato común: no está en ninguna clave.

3

1FN, 2FN y 3FN

Pruebas basadas en la clave principal y su descomposición

Formas normales: la escalera



Reunión no aditiva — obligatoria

Al unir las partes no salen filas inventadas.
Ej.: `ED1 ⋈ ED2` devuelve exactamente `EMP_DEPT`

Conservación de DF — deseable

Ninguna DF se pierde al partir; a veces se sacrifica.
Ej.: pasar `PARCELAS1A` a BCNF pierde FD2

Primera forma normal (1FN)

Cada celda guarda **un solo valor** (atómico): nada de listas, valores compuestos ni tablas anidadas

NombreDpto	NúmeroDpto	DniDirector	UbicacionesDpto
Investigación	5	333445555	{Valencia, Sevilla, Madrid}
Administración	4	987654321	{Gijón}
Sede central	1	888665555	{Madrid}

No está en 1FN

1. Nueva relación ✓

`LOCALIZACIONES_DPTO(NúmeroDpto, UbicaciónDpto)`: sin redundancia, general

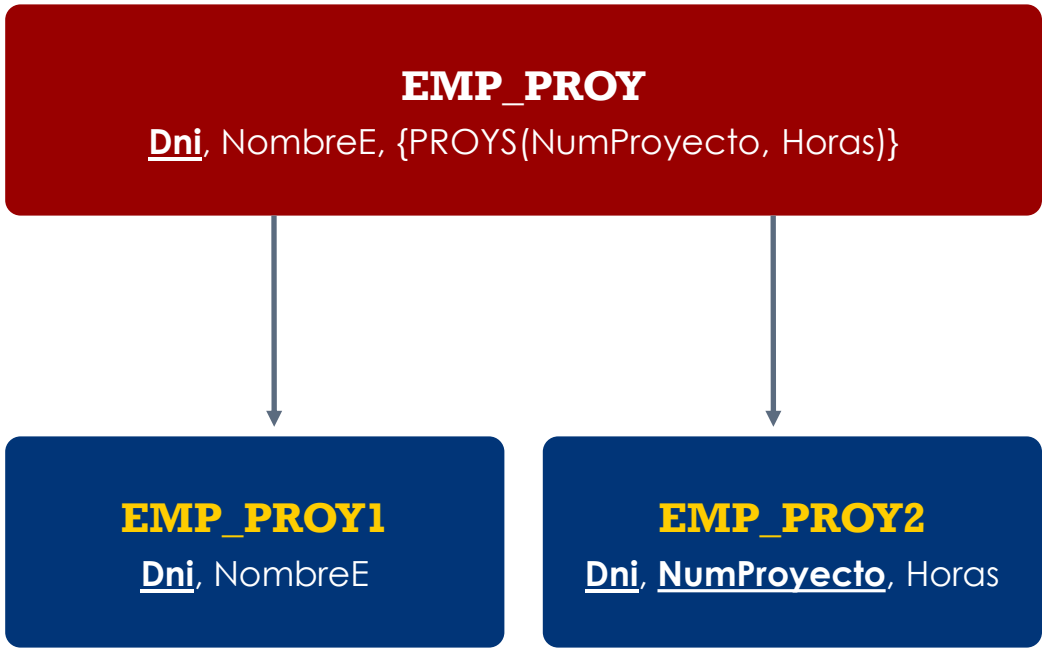
2. Expandir la clave

`{NúmeroDpto, UbicaciónDpto}`: una tupla por ubicación → **redundancia**

3. Columnas fijas

`UbicaciónDpto1..3`: máximo fijo, **NULL**, consultas difíciles

1FN: relaciones anidadas y varios multivalor



La clave Dni se propaga a la nueva relación

Varios atributos multivalor

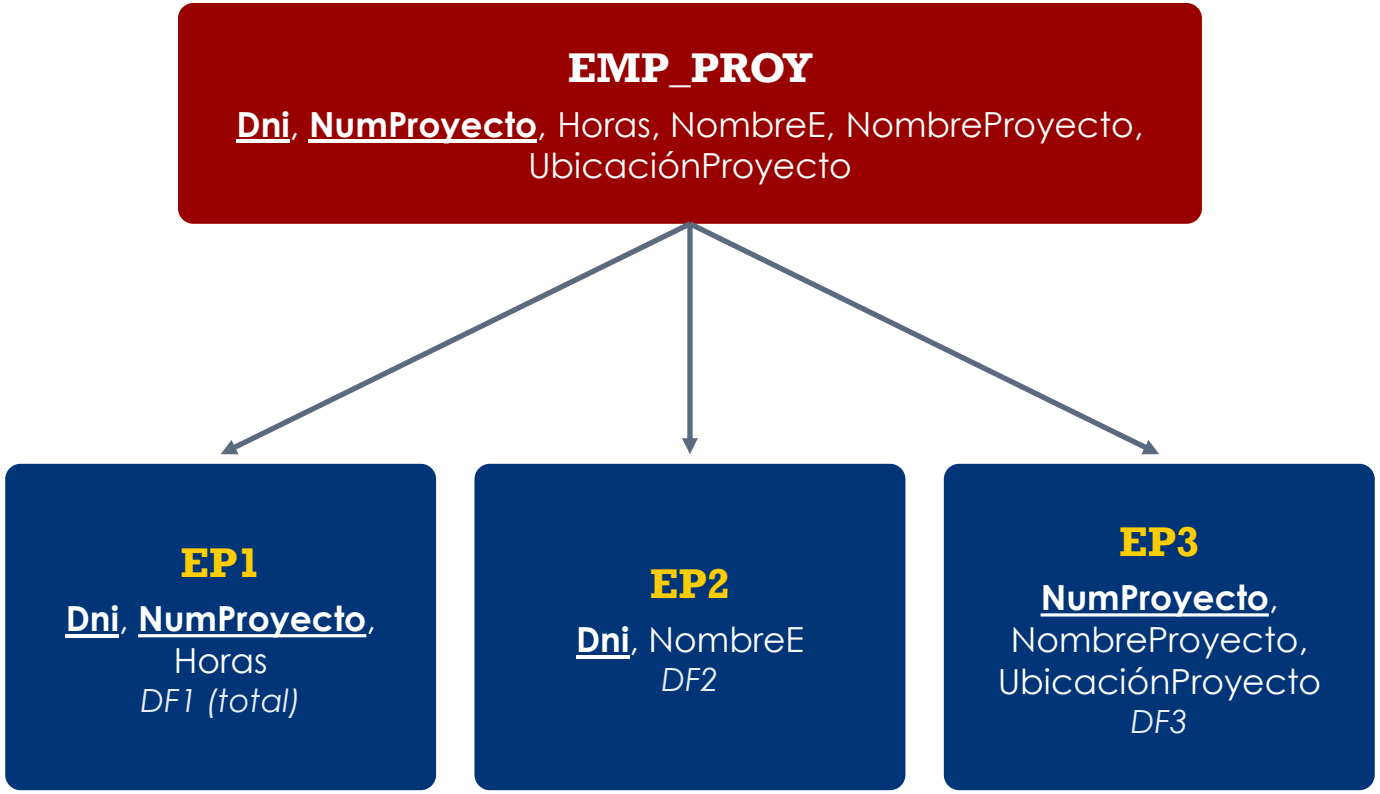
Una relación por cada uno:

```
PERSONA(Dn#, {NumPermiso#}, {Teléfono#})
↓
P1(Dn#, NumPermiso#)
P2(Dn#, Teléfono#)
```

Juntarlos en una sola relación genera redundancia: es el problema de la **4FN** (cap. 11).

Segunda forma normal (2FN)

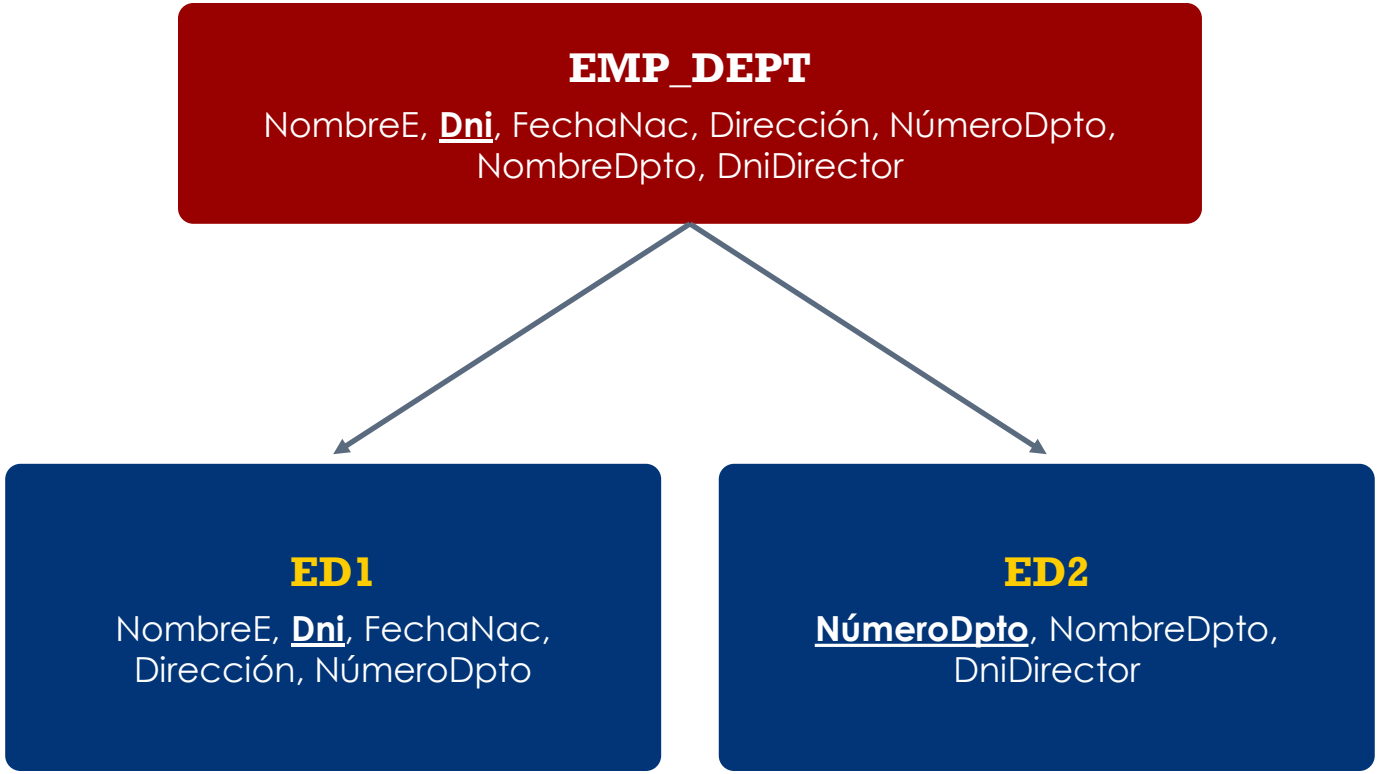
- **Dependencia total**: Y necesita **toda** la clave
  - `{Dni, NumProyecto} → Horas`
- **Dependencia parcial**: basta **un pedazo**
  - `{Dni, NumProyecto} → NombreE`
- **2FN**: cada dato depende de **toda** la clave
- Clave de **un** atributo ⇒ ya está en 2FN



Las DF parciales DF2 y DF3 salen en relaciones propias

Tercera forma normal (3FN)

- **Dependencia transitiva**: `X → Z → Y` con `Z` no clave
  - `Dni → NúmeroDpto → DniDirector`
- **3FN** (Codd): está en 2FN **y** ningún dato depende de **otro dato** que no es clave



`ED1 NATURAL JOIN ED2` recupera `EMP_DEPT` **sin tuplas falsas**

Resumen: 1FN – 3FN

Forma normal	Prueba	Remedio
1FN	Sin atributos multivalor ni relaciones anidadas	Nueva relación por cada atributo multivalor o relación anidada
2FN	(Clave compuesta) ningún atributo no clave depende de **parte** de la clave	Nueva relación por cada parte de la clave con sus dependientes; la original conserva lo que depende de la clave completa
3FN	Ningún atributo no clave depende de **otro atributo no clave**	Nueva relación con el atributo no clave y lo que determina

Definiciones generales de 2FN y 3FN

2FN general

Ningún atributo **no primo** depende **parcialmente** de **ninguna** clave candidata.
Ej.: `NombreMunicipio → Impuestos` viola 2FN: `NombreMunicipio` es parte de la clave `{NombreMunicipio, NúmeroParcela}`

Alternativa de 3FN

Todo atributo no primo depende **totalmente** de **cada** clave y **no transitivamente** de ninguna.
Ej.: `IdPropiedad → Área → Precio` es transitiva: viola 3FN

3FN general: para toda DF no trivial X → A

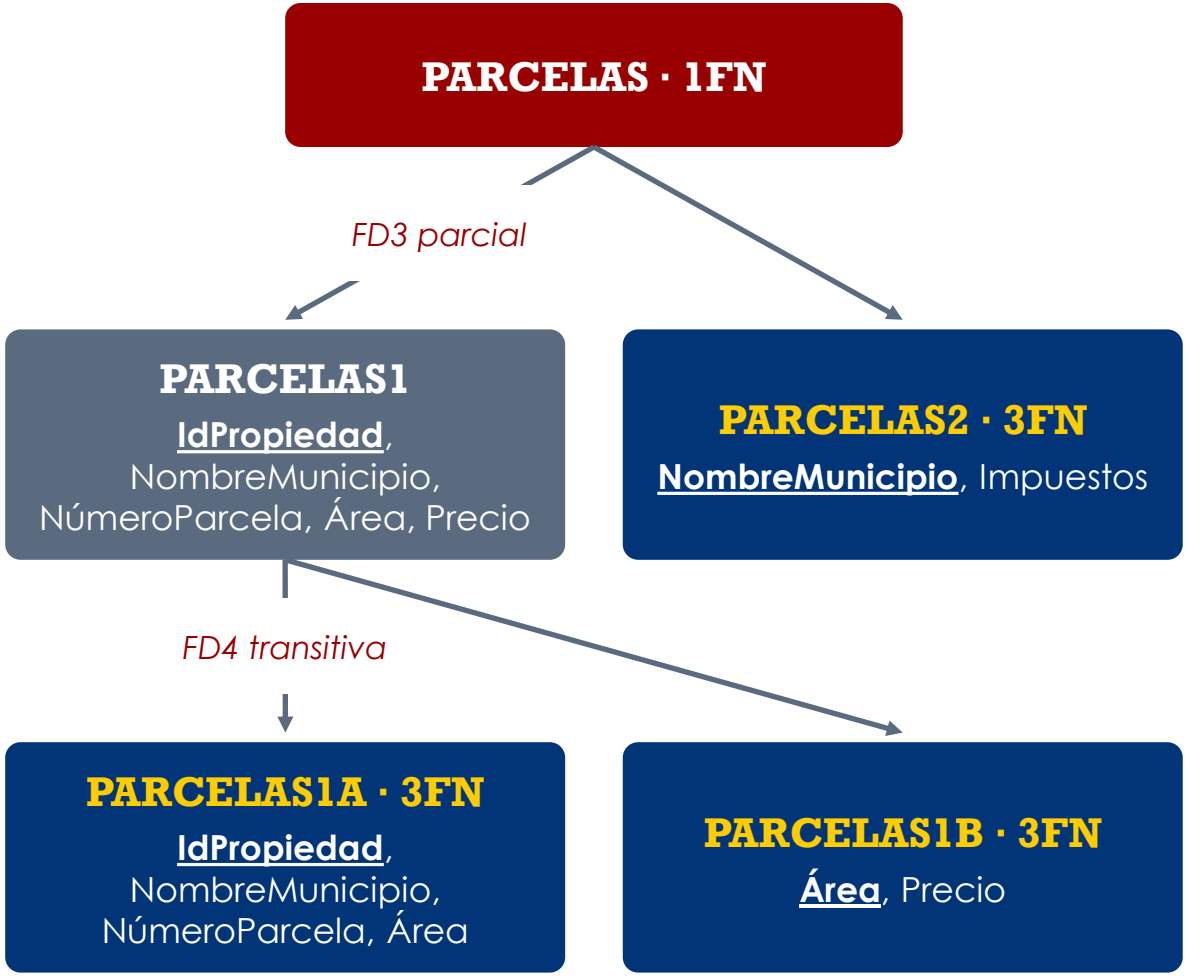
(a) X es superclave, o — Ej.: `IdPropiedad → Área`
(b) A es atributo primo — Ej.: `Área → NombreMunicipio (FD5, se ve en BCNF)`

Se aplica directamente, sin pasar antes por 2FN.

Ejemplo: PARCELAS

DF	Significado
FD1: `IdPropiedad → resto`	Clave candidata (principal)
FD2: `{NombreMunicipio, NúmeroParcela} → resto`	Otra clave candidata
FD3: `NombreMunicipio → Impuestos`	Impuesto fijo por municipio
FD4: `Área → Precio`	El precio depende del área

Se pueden eliminar en cualquier orden



4

Forma normal de Boyce-Codd

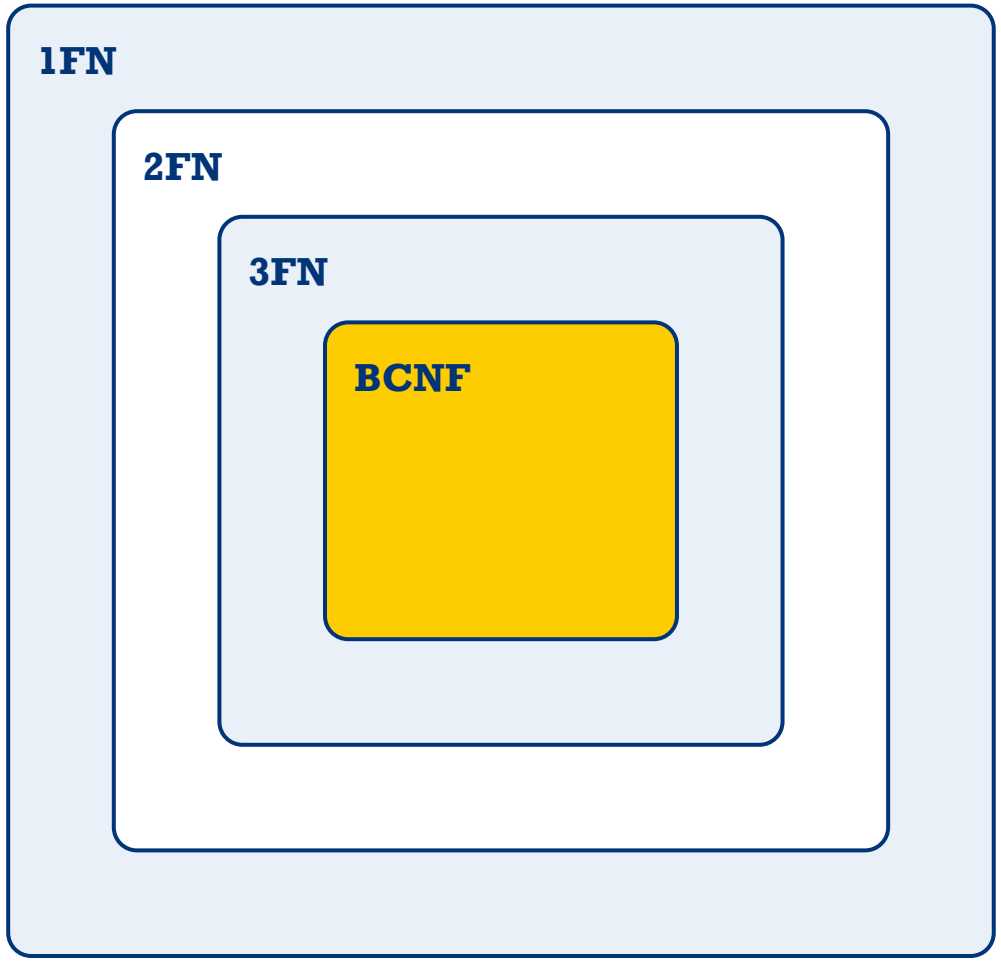
Solo una superclave puede decidir el valor de otra columna

BCNF: más estricta que 3FN

Para toda DF no trivial **X → A**

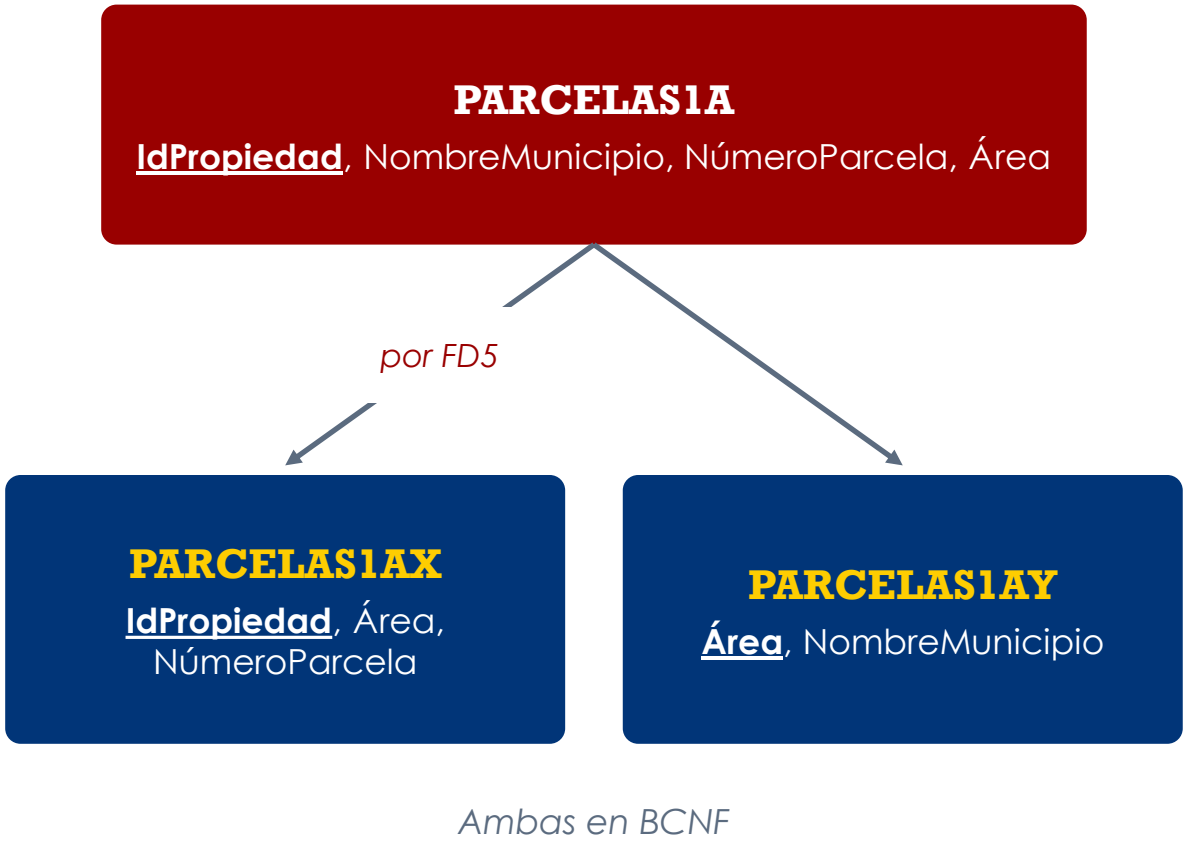
X es superclave

- Es la 3FN general **sin** la condición (b): ya no basta que A sea primo
- Toda relación en BCNF está en 3FN; **no** al revés
- En la práctica, casi todas las relaciones en 3FN ya están en BCNF



PARCELAS1A con FD5: 3FN pero no BCNF

- Parcelas solo en Getafe y Alcorcón, con áreas distintas → **FD5**: `Área → NombreMunicipio`
- Sigue en **3FN**: `NombreMunicipio` es primo
- **No** está en BCNF: `Área` no es superclave



Ambas en BCNF

Se **pierde FD2** `{NombreMunicipio, NúmeroParcela} → resto`: sus atributos ya no están juntos

ENSEÑAR: ¿cómo partirla?

Estudiante	Curso	Profesor
Campos	Bases de datos	Marcos
Pérez	Bases de datos	María
Pérez	Sistemas operativos	Amanda
Pérez	Teoría	Sergio
Ochoa	Bases de datos	Marcos
Ochoa	Sistemas operativos	Aurora
Morera	Bases de datos	Eduardo
Celaya	Bases de datos	María
Campos	Sistemas operativos	Amanda

```
FD1: {Estudiante, Curso} → Profesor
FD2: Profesor → Curso
```

3FN pero **no** BCNF: `Profesor` no es superclave

Descomposición	¿Tuplas falsas?
`{Estudiante, Profesor}` y `{Estudiante, Curso}`	Sí
`{Curso, Profesor}` y `{Curso, Estudiante}`	Sí
`{Profesor, Curso}` y `{Profesor, Estudiante}`	**No** ✓

Las tres pierden FD1

Resumen

1. **Directrices** Semántica clara, sin anomalías, pocos NULL, sin tuplas falsas
2. **DF** Restricción semántica; se infieren con los axiomas de Armstrong
3. **Escalera** 1FN atómico → 2FN sin parciales → 3FN sin transitivas → BCNF
4. **Meta** 3FN o BCNF; 1FN y 2FN son pasos intermedios

Siguiente: propiedades de la descomposición, 4FN y 5FN (cap. 11)